

SAVANNA SENTINEL



Deployment Requirements

Last Revision: 29 July 2026

David van Rooijen

Dandré Nel

Daniel Cohen

Stephan Kritzinger

Micheal Koch

TABLE OF CONTENTS

Deployment.....	2
Environments.....	2
Infrastructure.....	2
First-time setup.....	3
1.AWS - IAM and ECR.....	3
2. AWS Security Group.....	3
3.Launch.....	3
4.Point DNS at the instance.....	3
5.Install Docker.....	4
6.Clone and Configure.....	4
7.First Launch.....	6
Secrets.....	6
Rollback.....	6
Changelog.....	7
V1.....	7
V2.....	7
V3.....	7

Deployment

Savanna Sentinel runs on a single AWS EC2 instance, with the full stack under Docker Compose. This covers how it's deployed, what's needed to stand up a new instance, how the CD pipeline works, and how to roll back.

Environments

Environment	Where it runs	URL
Development	Local machine, <code>docker compose up</code>	<code>http://localhost:5173</code>
Production	AWS EC2, <code>docker compose --profile prod up</code>	<code>https://savannasentinel.co.za/</code>

No separate staging environment exists. Pull requests with required CI checks (`'backend-check'`, `'frontend-check'`, `'e2e-tests'`) are the gate before anything reaches `'main'`. A push to `'main'` triggers `'pipeline.yml'`, which builds/pushes both images and then deploys, in that order.

Infrastructure

- One EC2 instance (Ubuntu 24.04, t3.small) with an Elastic IP, , `'eu-north-1'`, 30GB gp3 root volume.
- Services: `db` (Postgres+PostGIS), `redis`, `seaweed`, `backend`, `worker`, `frontend`, `caddy`. All on one Docker network (`'savanna-sentinel_default'`).
- `caddy` is the only service exposed on ports 80/443 - everything else stays on the internal Compose network.
- TLS comes from Let's Encrypt via Caddy, for whatever hostnames `SITE_ADDRESS` and `MEDIA_SITE_ADDRESS` are set to. The domain is `'savannasentinel.co.za'` (`'www'` included) plus `'media.savannasentinel.co.za'` for serving report/tip-off photos out of SeaweedFS, with `A` records at the registrar pointing all three at the instance's Elastic IP - Caddy requests and renews certificates for them automatically on container start.

First-time setup

1.AWS - IAM and ECR

- Two ECR repositories: `savanna-sentinel-backend`, `savanna-sentinel-frontend`.
- An OIDC identity provider trusting `token.actions.githubusercontent.com`, so GitHub Actions can assume roles without any long-lived AWS keys stored as secrets.
- A push role, assumable by GitHub Actions, scoped to `ecr:PutImage / ecr:InitiateLayerUpload / ecr:UploadLayerPart / ecr:CompleteLayerUpload / ecr:BatchCheckLayerAvailability / ecr:BatchGetImage` on the two repos above, plus `ecr:GetAuthorizationToken` on `*`.
- A deploy role, assumable by GitHub Actions, scoped only to `ssm:SendCommand / ssm:GetCommandInvocation` against the instance and the AWS-RunShellScript document. It has no ECR permissions at all - the instance authenticates to ECR itself.
- An EC2 instance profile (attached directly to the instance, not assumed by GitHub) with the `AmazonSSMManagedInstanceCore` managed policy plus `ecr:BatchGetImage / ecr:GetDownloadUrlForLayer / ecr:BatchCheckLayerAvailability` on the two repos and `ecr:GetAuthorizationToken` on `*`.

2. AWS Security Group

Inbound: `80`, `443` from `0.0.0.0/0`. Outbound: default allow-all.

3.Launch

Ubuntu Server 24.04 LTS, `t3.small`, 30GB gp3 storage, using the security group above and the instance profile from step 1. Allocate an Elastic IP and associate it with the instance.

It should then show up as "Online" under Systems Manager -> Fleet Manager in the console.

4.Point DNS at the instance

At the domain registrar, create:

- A record: `@` -> `<elastic-ip>`
- CNAME record: `www` -> `<elastic-ip>`
- A record: `media` -> `<elastic-ip>`

Give it time to propagate before continuing, Caddy's certificate request in step 7 will fail if the domain doesn't resolve to this instance yet. Check with:

```
dig +short savannasentinel.co.za
```

It should return only the Elastic IP, nothing else.

5.Install Docker

Temporarily add port 22 at inbound security group and get a ec2 keypair to be able to ssh into instance.

Remove port 22 once finished.

```
ssh -i your-key.pem ubuntu@<elastic-ip>
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
# log out and back in for the group change to apply
```

6.Clone and Configure

```
git clone https://github.com/COS301-SE-2026/Savanna-Sentinel.git
cd Savanna-Sentinel
```

Generate a JWT Secret:

```
openssl rand -hex 32
```

Root `.env` from `.env.example`:

```
cp .env.example .env
nano .env
```

```
POSTGRES_USER=sentinel
POSTGRES_PASSWORD=<strong password>
POSTGRES_DB=savanna_sentinel
MINIO_ACCESS_KEY=sentinel_minio
MINIO_SECRET_KEY=<strong password>
SITE_ADDRESS=savannasentinel.co.za www.savannasentinel.co.za
MEDIA_SITE_ADDRESS=media.savannasentinel.co.za
```

backend/.env from backend/.env.example:

```
cp backend/.env.example backend/.env
nano backend/.env
```

```
DATABASE_URL=postgresql+asyncpg://sentinel:<same
POSTGRES_PASSWORD>@db/savanna_sentinel
JWT_SECRET=<paste the openssl output>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_SECONDS=3600
REFRESH_TOKEN_EXPIRE_DAYS=7
REDIS_URL=redis://redis:6379/0
MINIO_ENDPOINT=seaweedfs:9000
MINIO_PUBLIC_ENDPOINT=media.savannasentinel.co.za
MINIO_ACCESS_KEY=sentinel_minio
MINIO_SECRET_KEY=<same MINIO_SECRET_KEY>
MINIO_BUCKET=savanna-sentinel-media
MINIO_USE_SSL=true
MINIO_REGION=us-east-1
RESEND_API_KEY=<Resend API key>
RESEND_FROM_ADDRESS=noreply@savannasentinel.co.za
FRONTEND_BASE_URL=https://savannasentinel.co.za
MFA_ENABLED=true
```

7.First Launch

Run the same script the CD pipeline uses, so first launch and every later deploy go through identical code - pick a commit SHA that's already been built and pushed to ECR.

```
bash infra/deploy.sh <commit-sha> eu-north-1
```

Verify: `curl https://savannasentinel.co.za/v1/health` should return `{"status":"ok"}`.

Secrets

Set under repo Settings -> Secrets and variables -> Actions -> Secrets:

Secret	Value
<code>AWS_REGION</code>	<code>eu-north-1</code>
<code>AWS_ECR_PUSH_ROLE_ARN</code>	ARN of the push role from first-time setup step 1
<code>AWS_SSM_DEPLOY_ROLE_ARN</code>	ARN of the deploy role from first-time setup step 1
<code>EC2_INSTANCE_ID</code>	The instance ID
<code>EC2_APP_DIR</code>	<code>/home/ubuntu/Savanna-Sentinel</code>

Rollback

1. GitHub -> Actions -> "Deploy to production" -> Run workflow.
 2. Set `ref` to the commit SHA (or tag) to redeploy.
 3. Run it - the server pulls and runs the already-built image for that commit.
- This is a plain redeploy-previous-commit.